

PromptLens

Product Requirements Document + Architecture

Status: MVP proposal

Target release: AWS Deploy Your First App Weekend Challenge

Owner: AI Engineering

1. Product Requirements Document

1.1 Product summary

PromptLens is a lightweight prompt-regression testing workspace for AI engineers. It compares two prompt versions against the same saved test cases, invokes an Amazon Bedrock model, scores each result against a plain-English rubric, and highlights cases that improved, regressed, or need human review.

The MVP is deliberately narrow: one workspace, text-only test inputs and outputs, a selected Bedrock model, and rubric-based evaluation. It answers a practical release question: is this prompt change safe to ship?

1.2 Problem

Prompt changes are production changes, but engineers often assess them manually in chat windows. Comparisons become slow, inconsistent, difficult to reproduce, and easy to forget. A new prompt can improve a demo while quietly degrading important edge cases. Existing evaluation platforms can require substantial observability setup, datasets, pipelines, or enterprise procurement. Individual AI engineers and small teams need a focused way to test a prompt before merging it.

1.3 Target users

Primary user: an AI engineer or developer building an LLM feature with Amazon Bedrock.

Secondary users: technical founders, product engineers, and small teams who need a reviewable comparison before releasing a prompt update.

Not in MVP: annotation teams, multi-tenant enterprise administration, CI/CD gates, multimodal inputs, or fine-tuned-model evaluation.

1.4 Jobs to be done

- When I change a production prompt, I want to run it against representative cases so I can spot regressions before release.
- When results differ, I want side-by-side evidence and a concise explanation so I can decide which version is safer.
- When I find a useful or failing input, I want to save it as a durable regression case.

1.5 Goals and success metrics

Goals

1. Enable a first comparison run in under five minutes.

2. Make regressions obvious without reading every output.
3. Preserve enough metadata for a result to be reproducible.
4. Demonstrate a real, live AWS serverless application.

MVP success metrics

- A user can create a suite, add at least three cases, and complete a comparison run.
- Every case is labeled Improved, Regressed, Unchanged, or Needs review.
- A run stores prompt versions, model ID, inference settings, evaluator rubric, timestamps, raw outputs, and scores.
- A 3-case demo run completes reliably in under 60 seconds under normal Bedrock latency.

1.6 Core user flow

1. User creates a suite, for example “Customer-support replies.”
2. User adds input cases and an optional expected-behavior note.
3. User pastes baseline and candidate prompts, selects a Bedrock model, and writes an evaluator rubric.
4. User starts a comparison run.
5. PromptLens runs both prompts for every case, evaluates outputs, and assigns a status.
6. User reviews a summary, opens cases for side-by-side evidence, and saves useful cases to the suite.

1.7 MVP functional requirements

A. Prompt suites and cases

- Create, view, rename, and delete a suite.
- Add, edit, and delete text-only test cases.
- Each case has input, optional expected behavior, optional tags, and a stable case ID.
- Seed the app with one example suite so visitors can run a demo immediately.

B. Comparison configuration

- Enter baseline and candidate prompt text.
- Select one allowlisted Bedrock model.
- Set temperature and maximum output tokens.
- Enter a natural-language rubric, for example: “The answer must be concise, accurate, empathetic, and must not promise unsupported actions.”

C. Run execution

- Execute baseline and candidate prompts against each selected case.
- Run a separate evaluator invocation for each output using the rubric and expected behavior.
- Store numerical score from 1 to 5, evaluator rationale, output text, timing, and errors.
- Return partial results if an individual case fails.

D. Results and review

- Show run-level counts for improved, regressed, unchanged, needs-review, and failed cases.
- Show case-level baseline/candidate outputs, scores, and evaluator rationales side by side.

- Candidate score lower = Regressed; higher = Improved; equal = Unchanged. Evaluator error, low confidence, or material output difference with equal score = Needs review.
- Filter results by status and test-case tags.

E. Safety and guardrails

- Cap a MVP run at 20 cases.
- Cap prompt and input length to control cost and latency.
- Never expose AWS credentials to the browser.
- Allow only server-configured Bedrock model IDs.
- State clearly that AI evaluation is decision support, not a replacement for human review.

1.8 Non-functional requirements

Performance: initial UI load under 3 seconds on typical broadband; 3-case run target under 60 seconds.

Security: least-privilege IAM, HTTPS-only endpoints, and CORS restricted to the Amplify domain.

Privacy: avoid real customer data. Warn users not to enter sensitive personal, health, payment, credential, or confidential production data.

Observability: structured Lambda logs with run ID and case ID, plus CloudWatch metrics and alarms for API errors and invocation failures.

1.9 Out of scope for the weekend MVP

Authentication and collaboration; CI integration; webhooks; scheduled evaluation; dataset uploads; PDF/image/audio testing; statistical benchmarking; vector search; model routing; custom fine-tuning; and autonomous agents.

1.10 Future roadmap

V1: Cognito authentication, shareable run links, CSV export, saved rubric templates, and asynchronous runs.

V2: GitHub pull-request integration, API access, custom assertions, version history, and nightly regression runs.

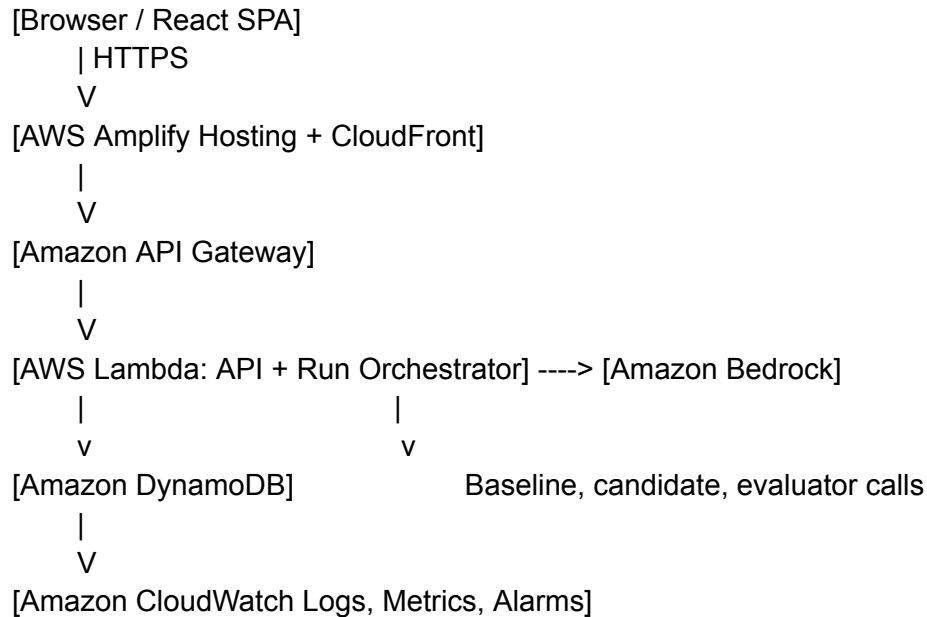
V3: human annotations, organization workspaces, role-based access, cost analytics, and CI quality gates.

2. Architecture Document

2.1 Architecture overview

PromptLens uses a serverless AWS architecture. The frontend is hosted on AWS Amplify. It calls an HTTPS API exposed by Amazon API Gateway. AWS Lambda validates requests, persists data in DynamoDB, orchestrates Amazon Bedrock invocations, and returns normalized results to the client. Amazon CloudWatch captures logs and operational metrics.

Architecture diagram



2.2 AWS services

AWS Amplify Hosting: builds and hosts the React single-page application with HTTPS and CDN distribution.

Amazon API Gateway HTTP API: public API boundary for suite, case, and run operations.

AWS Lambda: validates requests, enforces limits, invokes Bedrock, calculates comparison statuses, and reads/writes DynamoDB.

Amazon Bedrock: runs the selected foundation model for baseline/candidate generation and structured evaluator scoring.

Amazon DynamoDB: stores suites, cases, run metadata, raw outputs, scores, and evaluator rationales.

Amazon CloudWatch: captures logs, metrics, dashboards, and alarms.

AWS IAM: enforces least-privilege permissions between Lambda, DynamoDB, Bedrock, and CloudWatch.

2.3 Frontend

Technology: React, TypeScript, and Vite deployed through Amplify.

Key screens

- Home: lists suites and opens the seeded demo.
- Suite editor: test-case list plus baseline/candidate prompt editor.
- Run configuration: model, temperature, token limit, rubric, and selected cases.
- Results: summary cards, status filters, and case-comparison detail.

The frontend stores no AWS secret, Bedrock credential, or unrestricted model configuration. It reads only the API base URL from a build-time environment variable.

2.4 API design

GET /suites: list suites
POST /suites: create suite
GET /suites/{suiteId}: read suite and cases
PATCH /suites/{suiteId}: rename suite
DELETE /suites/{suiteId}: delete suite
POST /suites/{suiteId}/cases: add case
PATCH /suites/{suiteId}/cases/{caseId}: edit case
DELETE /suites/{suiteId}/cases/{caseId}: delete case
POST /runs: start a comparison run
GET /runs/{runId}: retrieve run metadata and results

POST /runs request example

```
{
  "suiteId": "support-replies",
  "caseIds": ["c-001", "c-002"],
  "baselinePrompt": "...",
  "candidatePrompt": "...",
  "modelId": "configured-model-alias",
  "temperature": 0.2,
  "maxTokens": 400,
  "rubric": "Be accurate, empathetic, concise, and do not invent policy."
}
```

Result classification

- Improved: candidate score is greater than baseline score.
- Regressed: candidate score is lower than baseline score.
- Unchanged: scores are equal and evaluator confidence is acceptable.
- Needs review: evaluator cannot produce valid structured output, confidence is low, output is truncated, or configured material-difference rules trigger.
- Failed: generation or storage fails after bounded retries.

2.5 Data model

Use one DynamoDB table for the MVP, named PromptLens, with partition key PK and sort key SK. This keeps deployment small while supporting suite and run access patterns.

Suite: PK = SUITE#{suiteId}; SK = META; fields: name, createdAt, updatedAt.

Test case: PK = SUITE#{suiteId}; SK = CASE#{caseId}; fields: input, expectedBehavior, tags.

Run: PK = RUN#{runId}; SK = META; fields: suiteId, modelId, prompts, rubric, settings, status, createdAt.

Run-case result: PK = RUN#{runId}; SK = CASE#{caseId}; fields: baselineOutput, candidateOutput, scores, rationales, latency, classification, error.

A GSI is optional for the weekend MVP. If added, use a suiteId index to list run history for a suite. Limit write sizes and truncate oversized model output with an explicit indicator so items stay under DynamoDB limits.

2.6 Lambda execution design

The first implementation runs synchronously for up to three cases. It creates the clearest live demo and keeps the code small. For each case, Lambda:

1. Validates limits and resolves the model alias to an allowed Bedrock model ID.
2. Builds generation requests for baseline prompt plus input and candidate prompt plus the same input.
3. Calls Bedrock for both outputs.
4. Calls a Bedrock evaluator with strict JSON instructions, output, rubric, and expected behavior.
5. Validates evaluator JSON, calculates status, and writes a run-case record.
6. Returns collected results, including per-case errors.

For longer runs, evolve to asynchronous execution: API Gateway creates a run, sends case jobs to SQS, worker Lambda processes jobs, and the client polls GET /runs/{runId}. This avoids API timeout pressure and improves resilience.

2.7 Bedrock prompt and evaluation design

Use the same selected model for baseline and candidate outputs in a run so the comparison is meaningful. A cheaper allowlisted model may evaluate outputs, but PromptLens must record the evaluator model ID.

Evaluator response schema

```
{
  "score": 1,
  "confidence": "high",
  "rationale": "...",
  "violations": ["..."]
}
```

The evaluator prompt must assess only supplied evidence, avoid inventing policy or facts, and state uncertainty. Lambda validates the schema, clamps score values to 1 through 5, and marks malformed results as Needs review rather than silently trusting them.

2.8 Security and cost controls

- Lambda IAM permits only specific DynamoDB table actions and Bedrock invocation actions for approved model resources.
- API Gateway validation and Lambda-side validation enforce maximum cases, input size, output tokens, and rubric length.
- No access keys are stored in frontend code or DynamoDB.
- Enable limited CloudWatch log retention. In production, log IDs, sizes, durations, status, and errors rather than full prompts or outputs.

- Start with a small Bedrock model and conservative output-token maximum. Show an estimated invocation count before a run starts.
- Before public launch, add Amazon Cognito or equivalent access control to prevent unbounded public Bedrock spending.

2.9 Reliability and failure handling

- Use bounded exponential-backoff retries only for retryable Bedrock errors.
- Persist a run as RUNNING before model calls, then COMPLETED, PARTIAL, or FAILED.
- Do not fail an entire run because one case errors. Surface case-specific errors and add retry in a later version.
- Return a user-safe error message and record technical detail in CloudWatch with a correlation run ID.

2.10 Deployment plan

1. Create DynamoDB table and IAM execution role with AWS SAM or AWS CDK.
2. Deploy API Gateway and Lambda to a DEV environment.
3. Configure Bedrock model access in the selected AWS Region.
4. Test API endpoints against seeded cases.
5. Push the React app to GitHub and connect the repository to AWS Amplify.
6. Set the Amplify API URL environment variable and publish.
7. Restrict API CORS to the Amplify URL.
8. Run the seeded demo end to end and capture screenshots for the Builder Center article.

2.11 Weekend build plan

Friday evening: scaffold React UI, DynamoDB table, Lambda, API endpoints, and one seeded suite.

Saturday: integrate Bedrock generation and structured evaluation; deploy API; confirm a complete three-case run.

Sunday: deploy frontend through Amplify; polish the results page; add error handling; capture architecture and demo screenshots; write the 500+ word challenge article.

2.12 Acceptance checklist

- [] Public Amplify URL loads successfully.
- [] Seeded suite contains at least three cases.
- [] Baseline and candidate prompts run against the same cases.
- [] Each completed case has two outputs, two scores, rationale, and a status.
- [] Results remain viewable after browser refresh.
- [] The app uses and documents Amplify, API Gateway, Lambda, DynamoDB, and Bedrock.
- [] Repository includes deployment instructions, environment-variable notes, and cost/safety guidance.